

Bus Servo ID Writing and Source Code Analysis

1. Servo Description

It is an automatic control system composed of a DC motor, reduction gears, a sensor, and control circuitry, which specifies the rotation angle of the output shaft by sending signals.

Differences between a servo and a motor:

A servo generally has a maximum rotation angle, while a DC motor rotates continuously. Therefore, a servo can only rotate within a certain angle and cannot rotate continuously like a DC motor. A regular DC motor cannot provide feedback on its rotation angle, whereas a servo can. Digital servos can switch between servo mode and motor mode.

2. Serial Bus Servo

A regular DC motor typically rotates continuously to provide power, while a servo is used to control the rotation of an object to a specific angle.

A bus servo is a derivative of a digital servo that communicates via an asynchronous serial bus, meaning it can be controlled by sending and receiving command packets, forming a closed-loop control system. Due to this feature, bus servos can be daisy-chained, making wiring simpler and reducing the use of serial ports. Because of this, when connecting bus servos, each servo's ID must be set in advance, or they cannot be individually controlled.

Bus servos use high-precision potentiometers internally, and unlike PWM servos, they can provide feedback on position, temperature, voltage, and other

information. Their accuracy and linearity are excellent, resulting in more stable robot operation and significantly extending the servo's lifespan.

Additionally, before assembling the servo, it must be set to its midpoint position. The servo midpoint refers to its initial position, considered the "zero point" from which it rotates to positive and negative angles. Therefore, the servo must be adjusted to the midpoint before being installed onto the servo horn.

This is because, during rotation, the moving parts drive the potentiometer. The software assumes the midpoint as the "zero point"; otherwise, the potentiometer might enter a "blind spot," causing the entire component to malfunction. Consequently, when using the robot, the specified angles may not be achieved, or the corresponding action sets may not match.

3. Procedure

- 1) We notice that the program defaults to setting the bus servo ID as 1, but this ID can be changed to other numbers.

```
33 serial_servo_set_id(0xFE, 1);
```

- 2) We can change the ID number to 5.

```
serial_servo_set_id(0xFE, 5);
```

- 3) After recompiling and burning the program, connect the servo debugging board to the computer's serial port. Open the Bus Servo Terminal software



, select the communication port, for example, COM15, click on parameter settings, and upon reading, it will show the ID number as 5, indicating that the ID has been successfully set.



2.5ms ----- 180 degrees

4. Code Analysis

File locations: Hiwonder/app.c void app_task_entry(void *argument)

Function

```
uint32_t broadcast_id = 0xFE; //It is the broadcast ID, and cannot be changed
uint8_t servo_id;
int ret = 0;

/* serial servo control initialization */
serial_servo_init();
```

At the beginning of the program, it initializes the hardware peripherals and creates relevant variables.

```
while(1)
{
    // Reading the servo ID can only be done with one servo connected. Otherwise, there will be a bus conflict
    ret = serial_servo_read_id(&serial_servo_controller, broadcast_id, &servo_id);
    if(ret == 0)
    {
        printf("servo id is %d\r\n", servo_id);
        break;
    }else{
        printf("read id fail, again\r\n");
    }
    osDelay(100);
}
```

It reads the ID of the servo connected to the bus. Only one servo can be connected to the bus to prevent conflicts. If the servo is successfully read, the program returns 0 and controls the servo to move to angle 0 ($0 \times 0.24^\circ = 0^\circ$), and then exits the loop. Otherwise, the program will be stuck in this loop and report an error log.

```
serial_servo_set_id(&serial_servo_controller , ret_servo_id , servo_id_new);  
printf("set servo new id is %d\r\n",servo_id_new);
```

Set new ID for the servo and print it to the serial port.

```
while(1)  
{  
    // Reading the servo ID can only be done with one servo connected. Otherwise, there will be a bus conflict  
    ret = serial_servo_read_id(&serial_servo_controller , broadcast_id , &servo_id);  
    if(ret == 0)  
    {  
        printf("servo id is %d\r\n",servo_id);  
        break;  
    }else{  
        printf("read id fail, again\r\n");  
    }  
    osDelay(100);  
}
```

After setting, the program reads the servo's ID again to verify that the new ID has been successfully written. If the read fails, the program will be stuck in this loop. If the read is successful, the program compares the read new ID with the preset new ID. If they are the same, the write is successful. Otherwise, the write fails, and the program is stuck in the next loop.

If the write is successful, the program controls the servo to move to angle 750 ($750 \times 0.24^\circ = 180^\circ$) with its new ID.

```
serial_servo_stop(&serial_servo_controller , servo_id); // make the servo stop  
  
// Loop: The loop in the RTOS task must have osDelay or other system blocking functions, otherwise it will cause system error  
for(;;)  
{  
    osDelay(300);  
}
```

After that, the program stops the servo's movement and completes its mission. It releases system resources in the for loop.

The `serial_servo_set_id()` function in the `Hiwonder/serial_servo.c` file is used to modify the ID of a specific servo from `old_id` to `new_id`. The servo ID is sent

to the servo through a serial data frame defined by the buf[] array.

```

48 void serial_servo_set_id(uint8_t old_id, uint8_t new_id) {
49     uint8_t buf[7];
50     buf[0] = buf[1] = SERIAL_SERVO_FRAME_HEADER;
51     buf[2] = old_id;
52     buf[3] = 4;
53     buf[4] = SERIAL_SERVO_ID_WRITE;
54     buf[5] = new_id;
55     buf[6] = CheckSum(buf);
56     SerialWrite(buf, 7);
57 }

```

The serial_servo_set_position() function sets the position and duration of the servo's movement. The specific position and duration information is sent to the servo through a serial data frame defined by the buffer[] array.

```

void serial_servo_set_position(SerialServoControllerTypeDef *self, uint32_t servo_id, int position, uint32_t duration)
{
    SerialServoCmdTypeDef frame;
    position = position > 1000 ? 1000 : position;
    cmd_frame_init(&frame, servo_id, SERIAL_SERVO_MOVE_TIME_WRITE);
    frame.elements.args[0] = GET_LOW_BYTE(position);
    frame.elements.args[1] = GET_HIGH_BYTE(position);
    frame.elements.args[2] = GET_LOW_BYTE(duration);
    frame.elements.args[3] = GET_HIGH_BYTE(duration);
    cmd_frame_complete(&frame, 4);
    self->serial_write_and_read(self, &frame, true);
}

```

The serial_servo_stop() function sends a control command to stop the servo through the serial port.

The execution flow is as follows:

- (1) Define a buffer to store the sent data frame.
- (2) Fill in the content of the data frame: frame header, servo ID, data length, stop servo, checksum, and finally send the data frame through the serial port.

```
118 void serial_servo_stop(uint8_t servo_id) {  
119     uint8_t buffer[10];  
120     buffer[0] = SERIAL_SERVO_FRAME_HEADER;  
121     buffer[1] = SERIAL_SERVO_FRAME_HEADER;  
122     buffer[2] = servo_id;  
123     buffer[3] = 6;  
124     buffer[4] = SERIAL_SERVO_MOVE_STOP;  
125     buffer[5] = CheckSum(buffer);  
126     SerialWrite(buffer, 6);  
127 }
```